

DV2627 Advanced ML

Lecture: Mining Data Streams

Veselka Boeva

AI & Data Analytics Lab, Department of Computer Science, BTH

veselka.boeva@bth.se

Outline

- Characteristics of data streams
- Data stream mining versus traditional mining
- Stream frequent pattern analysis
- Stream clustering analysis
 - STREAM algorithm (k -medians based)
 - CluStream algorithm (a framework for clustering evolving data streams)
- Clasification of data streams
 - Hoeffding tree
 - Very Fast Decision Tree (VSDT)
- References

Data streams

- Data evolve over time as new data arrive and old data become obsolete
- We can distinguish between:
 - Dynamic data arriving at a low rate
 - Incremental methods might work for such cases
 - Data streams: possible infinite sequence of elements arriving at a rapid rate
 - new methods are required to deal with the amount and complexity of these data

Characteristics of data streams

- Data Stream Model:
 - Data enters at a high-speed rate
 - The system cannot store the entire stream, but only a small fraction
 - How do you make critical calculations about the stream using a limited amount of memory?
- Characteristics
 - Huge volumes of continuous data, possibly infinite
 - Fast changing and requires fast, real-time response
 - Random access is expensive - single scan algorithms (*can only have one look*)

Examples of data evolution and concept drift in data streams

- Cluster Evolution

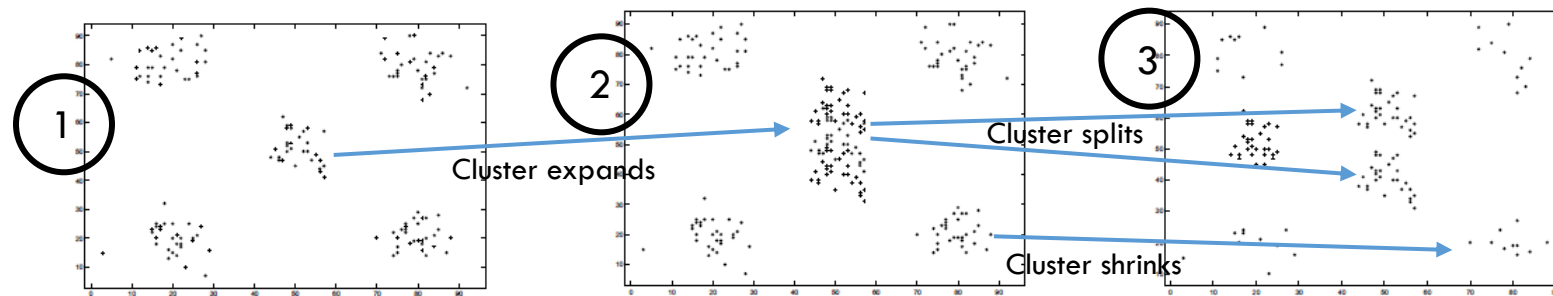
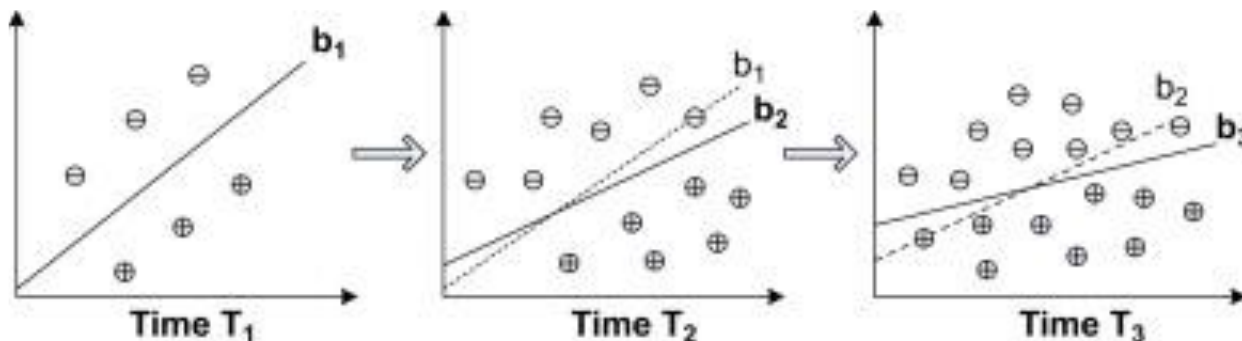


Figure: Data records at three consecutive time stamps, the clustering gradually changes (from: *MONIC -Modeling and Monitoring Cluster Transitions*, Spiliopoulou et al, KDD 2006)

- Concept Drift



An illustration of concept drifting in data streams. In the three consecutive time stamps T_1 , T_2 and T_3 , the classification boundary gradually drifts from b_1 to b_2 and finally to b_3 . (from: *A framework for application-driven classification of data streams*, Zhang et al, Journal Neurocomputing 2012)

Stream data application

- Telecommunication calling records
- Business: credit card transaction flows
- Network monitoring and traffic engineering
- Financial market: stock exchange
- Engineering & industrial processes: power supply & manufacturing
- Sensor, monitoring & surveillance: video streams, RFIDs (Radio-frequency identification)
- Web logs and Web page click streams
- Massive data sets (even saved but random access is too expensive)

Data stream mining versus traditional mining (1)

- Live streams *grow continuously*
 - The system cannot read streams many times as with traditional data mining
- Live streams mining must be done *on-line*
 - Not traditional data mining using off-line batch processing
- Live streams make mined results *changing*:
 - *Concept drift*: clusters and frequent itemsets continuously change over time as the data stream flow through the system

Data stream mining versus traditional mining (2)

- Live streams require processing very high data flow rates
 - *main memory processing* to keep up with high speed
- Live streams must be mined with *limited memory*
 - not load-and-analyze as traditional data mining
 - iterative processing of statistics
- Live streams must keep up with *very high data flow* volumes, using e.g.,
 - approximate mining algorithms
 - parallel on-line computations

Iterative Aggregation

- Data stream mining requires *iterative aggregation*:
 - Read data tuples *iteratively* from input stream(s), e.g., measured values
 - do not store read data in database
 - Analyze read tuples *incrementally*
 - Keep *partial results* in main memory
 - e.g., sum, count, min, max, standard deviation
 - Continuously *emit* incremental analyzed results
 - e.g., *running average* by dividing sum with count

Derived streams of summaries

- Result of data stream mining is *derived stream*
 - continuously produced by emitting tuples to output stream
 - *stream of statistical summaries* reduces the stream data rate
- *Continuously evolving* representation of mined data (drifting concepts):
 - *drifting concepts*: stream of mined summaries

Stored data

- Regular databases
 - if mined results grow continuously it may be not possible to store the mined data in database as with traditional data mining
 - current statistical summaries can be stored in continuously changing regular database
- Asynchronous *logging* in files often possible
 - at least of *low volume derived streams* of summaries
 - at least during *limited time* for raw data log files

One-pass processing

- Single scan of data
 - because of very large or infinite size of streams.
 - because it may be impossible or very expensive to reread the stream for the computation
- Limit memory and CPU usage during scan
 - because the processing should be done in main memory despite the very large stream volume

High data flow

- Stream processing should *keep up* with data flow
 - Make computations so fast that they keep up with the flow (on average)
 - Should not be catastrophic if miner cannot keep up with the flow:
 - Drop input data values when flow is too high
 - Use *approximated computations* such as sampling when flow is too high
 - Use *parallelism* when flow is too high

Data Stream Windows (1)

- Indefinite data size and limited memory
 - often mining over different kinds of *windows* (*chunks*) of streams
- A data stream window is a section of the stream
- Different kinds of windows of streams
 - e.g., sliding or tumbling windows
 - often statistics summaries (synopses, sketches) computed over the windows, e.g., moving average

Data Stream Windows (2)

- Need *window operator* to chop stream into segments
- Window *size* based on:
 - number of elements, a *counting* window, e.g., last 10 elements, i.e. windows have fixed size of 10 elements
 - a *time* window, e.g., last second elements, i.e., windows contain all events processed during the last second
 - a *landmark* window, e.g., all events from time t in a window

Data Stream Windows (3)

- Window based on a *decaying factor*
 - Decrease importance of measurement by multiplying with factor λ
 - Remove when importance below threshold
- Damped window:
 - Data are subject to ageing according to a fading function $f(t)$, i.e., each point is assigned a weight that decreases with time t via $f(t)$.
 - A widely used fading function in temporal applications is the exponential fading function: $f(t) = 2^{-\lambda t}$, $\lambda > 0$.
 - The decay rate λ determines the importance of historical data
 - The higher the value of λ , the lower the importance of old data

Data Stream Windows (4)

- Windows may also have *stride*
 - Rule for how fast they move forward, e.g.,
 - 10 elements for a 10-element counting window
 - a *tumbling* window
 - 2 elements for a 10-element counting window
 - a *sliding* window
 - 100 elements for a 10-element counting window
 - a *sampling* window
- Windows need not always be materialized
 - e.g., often sufficient to keep statistics materialized

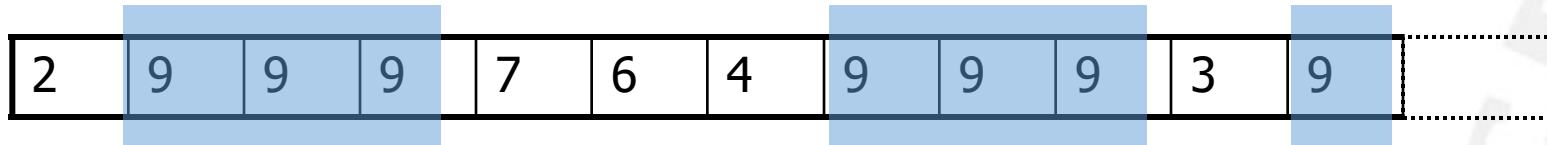
Mining Approximate Frequent Patterns

- Mining **precise** frequent patterns in stream data: **unrealistic**
 - Even store them in a compressed form, such as Frequent Pattern (FP) tree
- **Approximate answers** are often sufficient (e.g., trend/pattern analysis)
 - Example: a router is interested in all flows:
 - whose **frequency** is at least **1% (s)** of the entire traffic stream seen so far
 - and feels that **1/10 of s ($\epsilon = 0.1\%$) error** is comfortable
- How to mine frequent patterns with **good approximation**?
 - Based on Majority Voting - not tracing items until it becomes frequent
 - Lossy Counting Algorithm - identify elements whose frequency exceeds a user-given threshold

Mining Approximate Frequent Patterns

Majority Voting

- A sequence of N items
- You have constant memory
- In one pass, decide if some item is in majority (occurs $> N/2$ times)?



$N = 12$; item 9 is majority

Mining Approximate Frequent Patterns

Misra-Gries Algorithm

- A counter and an ID
 - If new item is same as stored ID, increment counter
 - Otherwise, decrement the counter
 - If counter 0, store new item with count = 1
- If counter > 0 , then its item is the only candidate for majority

	2	9	9	9	7	6	4	9	9	9	3	9
ID	2	2	9	9	9	9	4	4	9	9	9	9
count	1	0	1	2	1	0	1	0	1	2	1	2

Mining Approximate Frequent Patterns

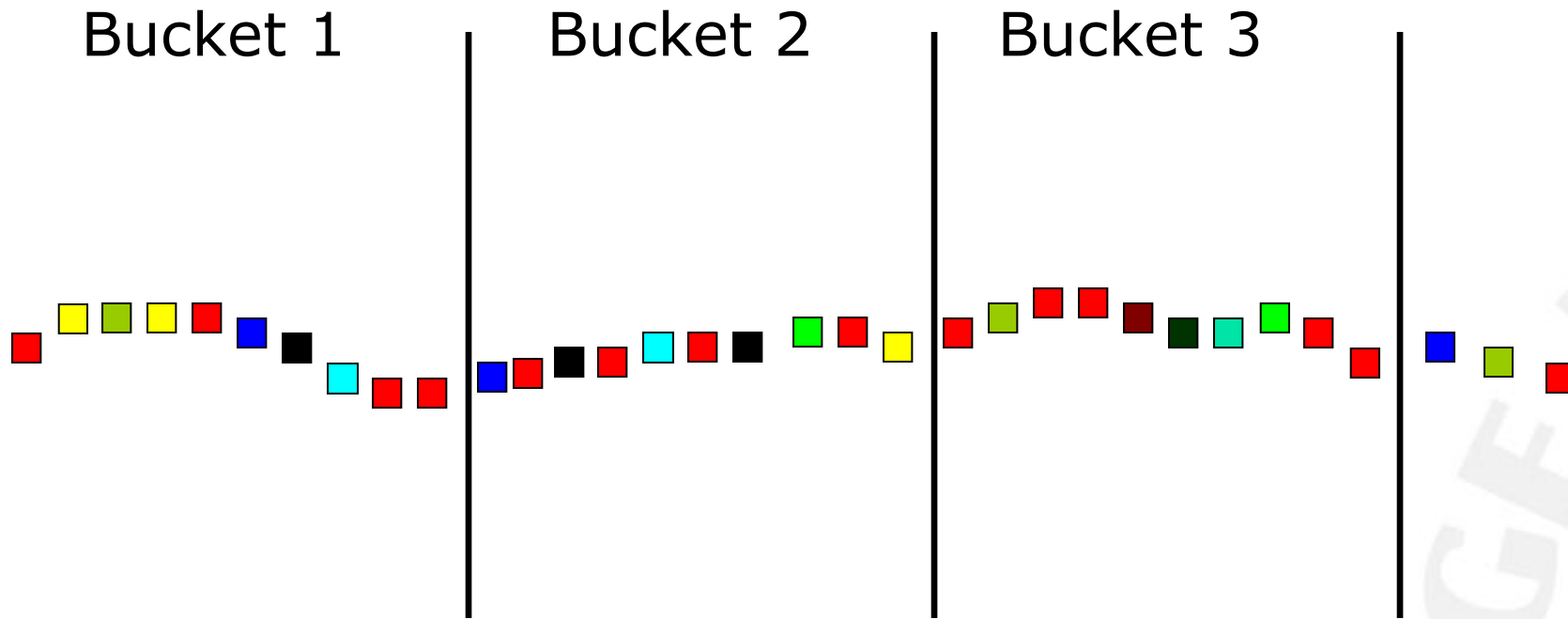
A generalization: Frequent Items

Find k items, each occurring at least $N/(k+1)$ times.

ID	ID ₁	ID ₂	.	.	.	.	ID _k
count	.	.					

- Algorithm:
 - Maintain k items, and their counters
 - If next item x is one of the k items, increment its counter
 - Else if there is a zero counter, put x there with count = 1
 - Else (all counters non-zero) decrement **all** k counters

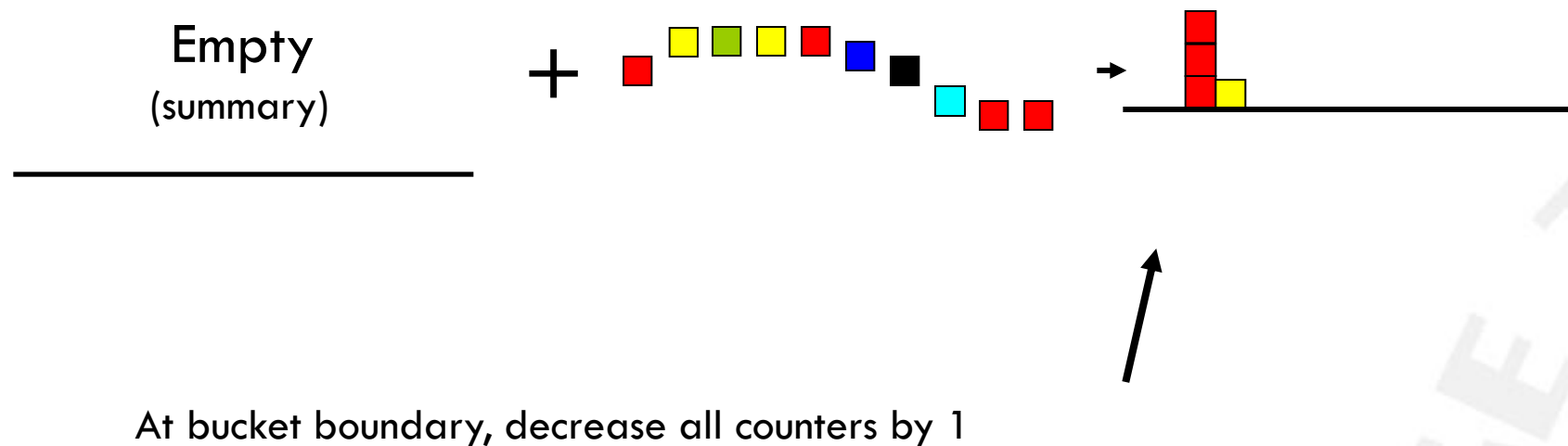
Lossy Counting for Frequent Items



Divide stream into 'buckets' (bucket size is $1/\epsilon = 1000$)

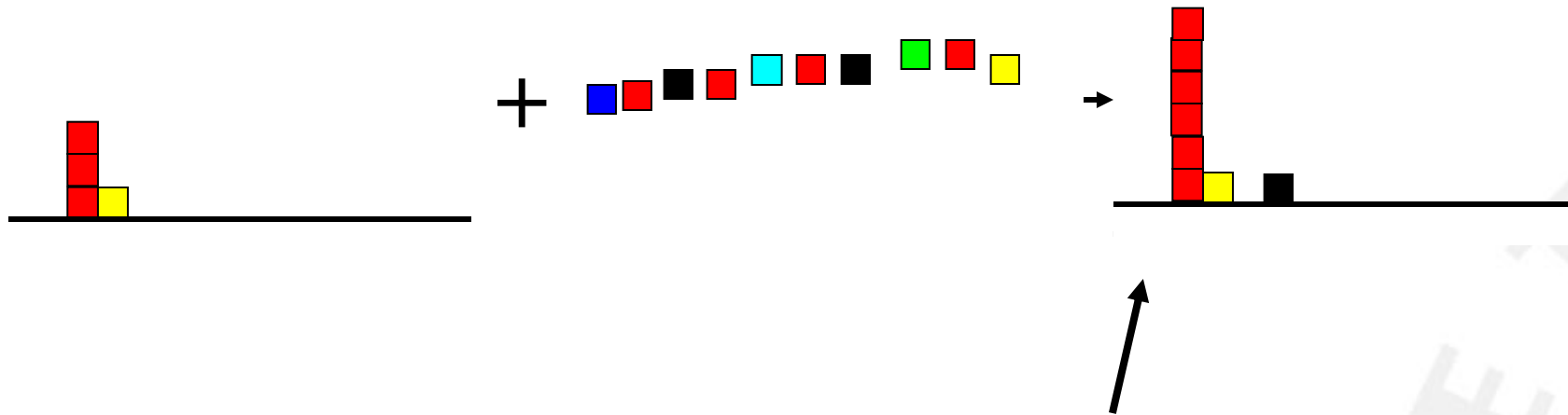
Lossy Counting for Frequent Items

First bucket of stream



Lossy Counting for Frequent Items

Next bucket of stream



At bucket boundary, decrease all counters by 1

Lossy Counting for Frequent Items

- Given: (1) support threshold: σ , (2) error threshold: ε , and (3) stream length N
- Output: items with frequency counts exceeding $(\sigma - \varepsilon)N$
- How much do we undercount?

If stream length seen so far = N , and bucket size = $1/\varepsilon$

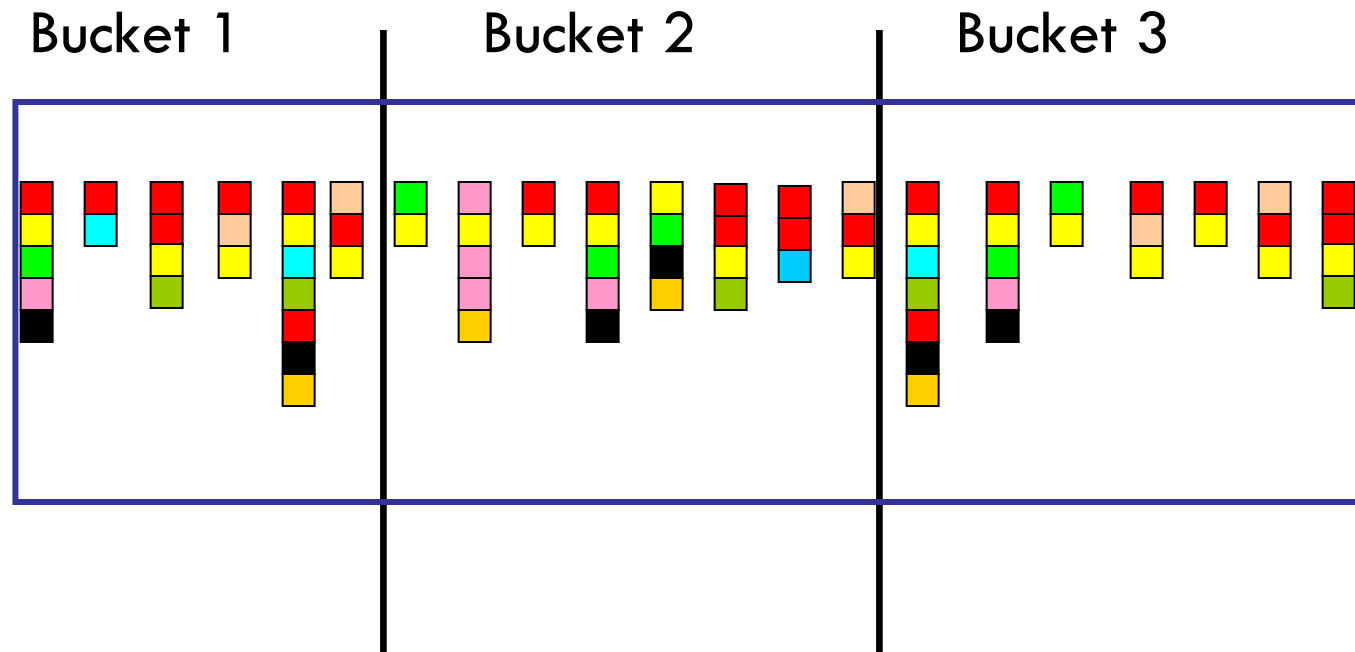
Then frequency count error $\leq \# \text{buckets} = \varepsilon N$

- **Approximation guarantee**

- No false negatives
- False positives have true frequency count at least $(\sigma - \varepsilon)N$
- Frequency count underestimated by at most εN

Lossy Counting For Frequent Itemsets

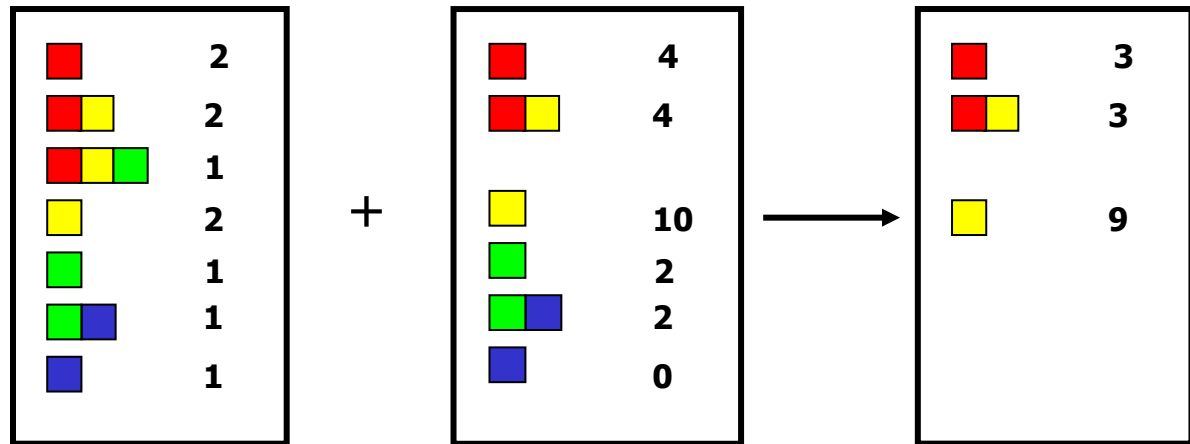
Divide stream into 'buckets' as for frequent items,
but fill as many buckets as possible in main memory one time



If we put **3 buckets of data** into main memory one time,
then **decrease each frequency count by 3**

Lossy Counting For Frequent Itemsets

Update of Summary Data Structure



summary data

3 bucket data
in memory

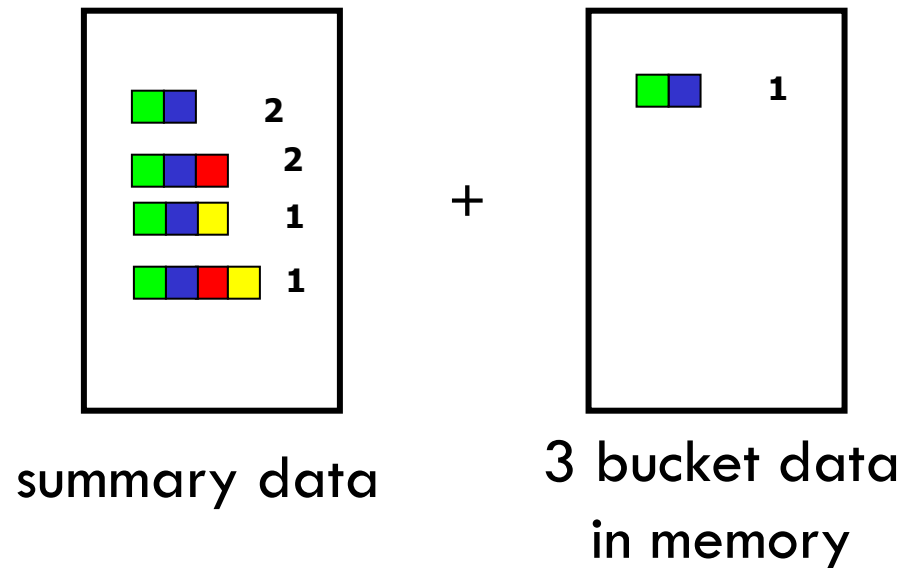
summary data

Itemset (Green, Blue) is deleted.

That is why we choose **a large number of buckets** – delete more

Lossy Counting For Frequent Itemsets

Pruning Itemsets – Apriori Rule



If we find itemset () is not frequent itemset,
Then we do not need to consider its superset

Summary of Lossy Counting

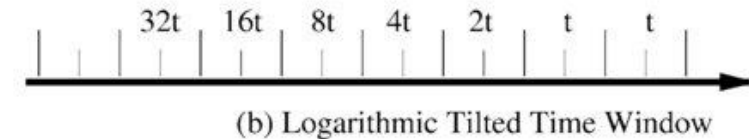
- Strength
 - A **simple** idea
 - Can be extended to frequent itemsets
- Weakness:
 - **Space bound** is not good
 - For frequent itemsets, they do **scan each record many times**
 - The output is based on **all previous data**. But sometimes, we are only interested in **recent data**

Mining Evolution of Frequent Patterns for Stream Data

- Approximate frequent patterns
 - Keep only current frequent patterns - No changes can be detected

- Mining evolution and dramatic changes of frequent patterns

- Use *tilted time window* frame
- Use compressed form to store significant (approximate) frequent patterns and their time-dependent traces



Note: To mine precise counts, one has to trace/keep a fixed (and small) set of items

Clustering data streams

- Goal: Construct a partition of a set of objects into k clusters
 - e.g., k -Means, k -Medoids
- Two types of methods:
 - Adaptive methods:
 - STREAM algorithm – based on median clustering methodology (S. Guha et al, 2003)
 - Online summarization -offline clustering methods:
 - CluStream (Aggarwal et al, 2003)

Clustering data streams – STREAM k -means algorithm

- Find k clusters in the stream so that the sum of distances from data points to their closest center is minimized
- An extension of an existing clustering for streams, e.g., k -means can be used
 - The iterative process of static k -means cannot be applied to streams
 - Use a buffer that fits in memory and apply k -means locally in the buffer
- Stream is processed in chunks, each fitting in memory
 - For each chunk k -means is applied locally and only the k centers are retained
 - Each center is treated as a point, weighted with the number of points it compresses
 - Apply k -means on the output containing the cluster centers that fit in the buffer

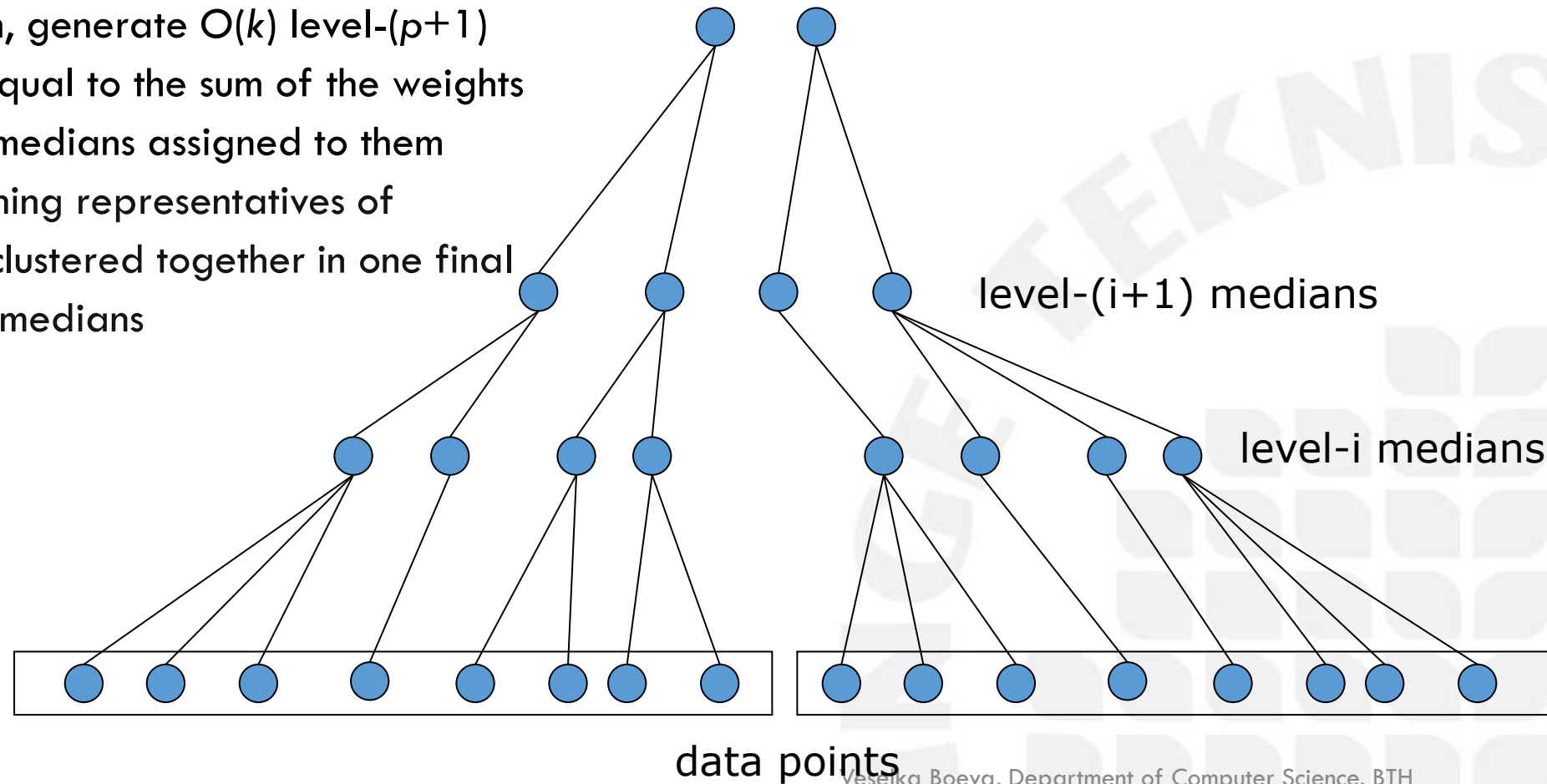
Clustering data streams – STREAM algorithm

- STREAM is a simple two step algorithm:
 1. For each set of m records, S_i , find $O(k)$ centers in stream chunks $S_1, \dots, S_r$
 - Local clustering by assigning each point in S_i ($i = 1, \dots, r$) to its closest center
 2. Let S' be centers for $S_1, \dots, S_r$ with each center weighted by number of points assigned to it
 - Cluster S' to find k centers

STREAM Algorithm - hierarchical clustering tree

Method:

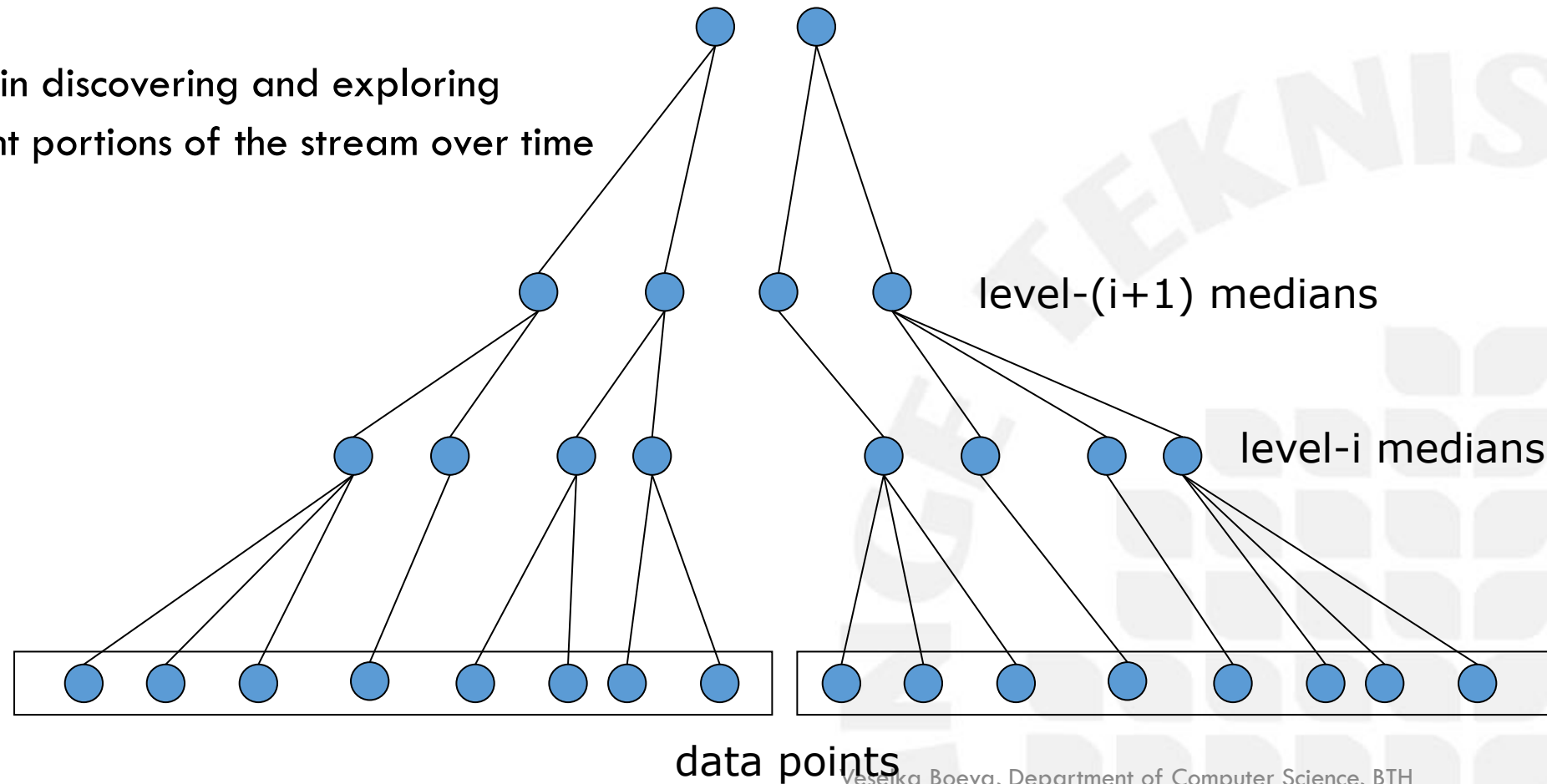
- Maintain at most m level- p medians
- On seeing m of them, generate $O(k)$ level- $(p+1)$ medians of weight equal to the sum of the weights of the intermediate medians assigned to them
- At the end all remaining representatives of different levels are clustered together in one final application of the k -medians



STREAM Algorithm - hierarchical clustering tree

Drawbacks:

- Low quality for evolving data streams (register only k centers)
- Limited functionality in discovering and exploring clusters over different portions of the stream over time



CluStream Algorithm

- Design goal
 - High quality for clustering evolving data streams with greater functionality
 - While keep the stream mining requirement in mind
 - One-pass over the original stream data
 - Limited space usage and high efficiency
- **CluStream:** A framework for clustering evolving data streams
 - Divide the clustering process into online and offline components
 - Online component (Microclustering): periodically stores summary statistics about the stream data
 - Offline component (Macroclustering): answers various user questions based on the stored summary statistics

The CluStream Framework

- Micro-cluster
 - Statistical information about data locality
 - Temporal extension of the *cluster-feature vector*
 - Multi-dimensional points $\bar{X}_1 \dots \bar{X}_k \dots$ with time stamps $T_1 \dots T_k \dots$
 - Each point contains d dimensions, i.e., $\bar{X}_i = (x_i^1 \dots x_i^d)$
 - A micro-cluster for n points is defined as a $(2.d + 3)$ tuple
$$(\overline{CF2^x}, \overline{CF1^x}, CF2^t, CF1^t, n)$$
- Pyramidal time frame
 - Decide at what moments the snapshots of the statistical information are stored away on disk

CluStream: Pyramidal Time Frame

- Pyramidal time frame
 - Snapshots of a set of micro-clusters are stored following the pyramidal pattern
 - They are stored at differing levels of granularity depending on the recency
 - Snapshots are classified into different orders varying from 1 to $\log(T)$
 - The i -th order snapshots occur at intervals of α^i where $\alpha \geq 1$
 - Only the last $(\alpha^i + 1)$ snapshots are stored

CluStream: Clustering On-line Streams

- Online micro-cluster maintenance
 - Initial creation of q micro-clusters
 - q is usually significantly larger than the number of natural clusters
 - Online incremental update of micro-clusters
 - If new point is within max-boundary, insert into the micro-cluster
 - otherwise, create a new cluster
 - May delete obsolete micro-cluster or merge two closest ones
- Query-based macro-clustering
 - Based on a user-specified time-horizon h and the number of macro-clusters k , compute macro-clusters using the k -means algorithm

Streaming Classification

- The main challenge is concept drift
- A simple approach is to use a reservoir sample to create a concise representation of the training data
 - The concise representation can be used to create an offline model.
 - A decay-based reservoir sample can be used to handle concept drift.
 - Any conventional classification algorithm can be used since the challenges of the streaming paradigm have already been addressed at the sampling stage.
- Dedicated methods have also been proposed for streaming classification
 - e.g., *Hoeffding trees*, *Very Fast Decision Trees (VFDT)* and *Concept-adapting VFDT*.

Classification with Hoeffding Tree

- With high probability, classifies tuples the same
- Only uses small sample
 - Based on Hoeffding Bound principle
- Hoeffding Bound (Additive Chernoff Bound)

r : random variable (splitting criteria)

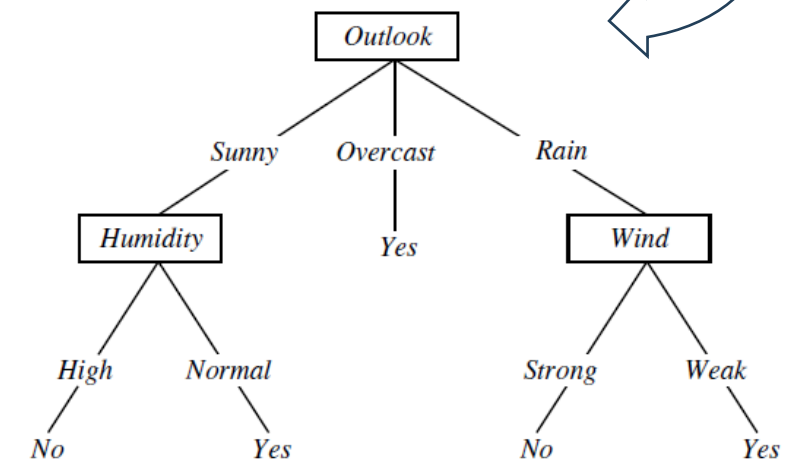
R : range of r

n : # independent observations

Mean of r is at least $r_{avg} - \epsilon$, with probability $1 - \delta$

$$\epsilon = \sqrt{\frac{R^2 \ln(1/\delta)}{2n}}$$

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No



Hoeffding Tree Algorithm

- Hoeffding Tree Input

S : sequence of examples

X : attributes

$G(\)$: evaluation function

d : desired accuracy

- Hoeffding Tree Algorithm

for each example in S

 retrieve $G(X_a)$ and $G(X_b)$ //two highest $G(X_i)$

 if ($G(X_a) - G(X_b) > \epsilon$)

 split on X_a

 recurse to next node

 break

Hoeffding Tree: Strengths and Weaknesses

- Strengths
 - Scales better than traditional methods
 - Sublinear with sampling
 - Very small memory utilization
 - Incremental
 - Make class predictions in parallel
 - New examples are added as they come
- Weakness
 - Could spend a lot of time with ties
 - Memory used with tree expansion
 - Number of candidate attributes

VFDT (Very Fast Decision Tree)

- Modifications to Hoeffding Tree
 - Near-ties broken more aggressively
 - G computed every n_{min}
 - Deactivates certain leaves to save memory
 - Poor attributes dropped
 - Initialize with traditional learner (helps learning curve)
- Compare to Hoeffding Tree: Better time and memory
- Compare to traditional decision tree
 - Similar accuracy
 - Better runtime with 1.61 million examples
 - 21 minutes for VFDT
 - 24 hours for C4.5
- Still does not handle concept drift

CVFDT (Concept-adapting VFDT)

- Concept Drift
 - Time-changing data streams
 - Incorporate new and eliminate old
- CVFDT
 - Increments count with new example
 - Decrement old example
 - Sliding window
 - Nodes assigned monotonically increasing IDs
 - Grows alternate subtrees
 - When alternate more accurate $\Rightarrow$ replace old
 - $O(w)$ better runtime than VFDT-window

References

- Charu C. Aggarwal (2015). Data Mining: The Textbook, Springer International Publishing Switzerland.
- G. Manku, R. Motwani. Approximate Frequency Counts over Data Streams, VLDB'02
- S. Guha, N. Mishra, R. Motwani, and L. O'Callaghan. Clustering Data Streams, FOCS'00
- C. Aggarwal, J. Han, J. Wang, P. S. Yu. A Framework for Clustering Data Streams, VLDB'03